

SIKSHA 'O' ANUSANDHAN

(Deemed To Be University)

Accredited (3rd Cycle) by NAAC with **A++ Grade**

Admission Batch: 2023 | Session: 2025–26

Major Project Report of

MACHINE LEARNING CONCEPT-2 (CSE 3968)

On

MLC Network Detection System

Network Anomaly Detection Using Machine Learning

IoT-23 Dataset · Isolation Forest · One-Class SVM · Autoencoder · Random Forest ·

Streamlit Deployment

Submitted by:

Name	Registration Number
NAME 1	2341_____
NAME 2	2341_____
NAME 3	2341_____
NAME 4	2341_____

Section: 2341_____ **Semester:** _____ **Branch:** Computer Science & Engineering

Department of Computer Science & Engineering
Institute of Technical Education and Research (ITER), S'O'A
Jagamohan Nagar, Jagamara, Bhubaneswar, Odisha – 751030

May, 2026

CERTIFICATE

This is to certify that the Major Project Report titled “**MLC Network Detection System: Network Anomaly Detection Using Machine Learning**” submitted by the students listed below has been carried out under the supervision of the undersigned and is a bonafide record of the work done by them in partial fulfilment of the requirements of the course **Machine Learning Concept-2 (CSE 3968)** for the academic session 2025–26.

Name	Registration No.
NAME 1	2341_____
NAME 2	2341_____
NAME 3	2341_____
NAME 4	2341_____

Supervisor / Faculty Guide

Name: _____

Designation: _____

Department: CSE, ITER, S'O'A

Head of Department

Name: _____

Designation: Professor & HOD

Department: CSE, ITER, S'O'A

Date: _____

Place: Bhubaneswar, Odisha

Abstract

The **MLC Network Detection System** is a machine learning-based network anomaly detection framework designed to identify whether network traffic is *Benign* or *Malicious*. As internet-connected devices and IoT systems grow rapidly, traditional signature-based Intrusion Detection Systems (IDS) become inadequate against novel and zero-day attack patterns. To address this limitation, this project employs supervised and unsupervised machine learning techniques on the **IoT-23 dataset**, a real-world collection of labeled network connection logs generated from IoT malware capture scenarios.

The project covers the complete machine learning pipeline: dataset loading, cleaning, Exploratory Data Analysis (EDA), feature engineering, preprocessing, model training, evaluation, model persistence, and deployment. A sample of **50,000 connection records** was selected for efficient experimentation while preserving statistical representativeness. Five categories of models were trained and compared — *Bagging*, *Boosting*, *Random Forest*, *Isolation Forest*, *One-Class SVM*, and a *deep learning Autoencoder*. The best-performing model was saved as a `.pkl` file using *Joblib/Pickle* and imported into a **Streamlit** web application.

Key Results: Random Forest achieved the highest independent model performance with **Accuracy: 0.9979**, **Precision: 0.9962**, **Recall: 1.0000**, and **F1-Score: 0.9981**. Among ensemble methods, Bagging (Decision Tree) and Gradient Boosting both reached perfect accuracy of 1.0000 on the test set. The deployed Streamlit application provides manual traffic prediction, row-level testing from `testing_data.csv`, interactive visualizations of byte/packet volume, port behavior, and prediction confidence — making the project practical, interactive, and presentation-ready.

Keywords: Network Anomaly Detection, IoT-23, Random Forest, Isolation Forest, One-Class SVM, Autoencoder, Ensemble Learning, Streamlit, Intrusion Detection, Feature Engineering, Machine Learning Pipeline.

Contents

Abstract	2
Table of Contents	5
List of Figures	6
List of Tables	7
1 Introduction	8
1.1 Background and Motivation	8
1.2 Machine Learning in Network Security	8
1.3 Project Overview	9
1.4 Why IoT-23 and Not KDD Cup 99?	9
1.5 Report Structure	9
2 Problem Statement	10
3 Objectives	11
4 Literature Review	12
4.1 Signature-Based and Rule-Based Systems	12
4.2 Machine Learning Approaches	12
4.2.1 Ensemble Methods	12
4.2.2 Anomaly Detection Models	12
4.2.3 Deep Learning Autoencoders	13
4.3 IoT-Specific Datasets	13
4.4 Summary of Gap Addressed	13
5 Dataset Description and Understanding	14
5.1 Dataset Used: IoT-23	14
5.2 Dataset Columns and Meanings	14
5.3 Target Variable Creation	15
5.4 Dataset Characteristics	16

6	Data Preprocessing	17
6.1	Raw Data Challenges	17
6.2	Loading and Initial Cleaning	17
6.3	Scikit-learn Preprocessing Pipeline	18
6.4	Train-Test Split	18
7	Exploratory Data Analysis (EDA)	20
7.1	Objectives of EDA	20
7.2	Missing Value Analysis	20
7.3	Traffic Class Distribution	21
7.4	Numeric Feature Distributions	21
7.5	Correlation Heatmap	21
7.6	EDA Intuition for Attack Detection	22
8	Feature Engineering	23
8.1	Why Feature Engineering Is Critical	23
8.2	Engineered Features and Justification	23
8.3	Feature Importance	25
9	Model Development and Justification	26
9.1	Model Selection Philosophy	26
9.2	Bagging Models	27
9.2.1	Bagging with Decision Tree	27
9.2.2	Random Forest	27
9.3	Boosting Models	27
9.3.1	AdaBoost	27
9.3.2	Gradient Boosting	27
9.3.3	HistGradientBoosting	27
9.4	Unsupervised Anomaly Detection Models	27
9.4.1	Isolation Forest	28
9.4.2	One-Class SVM	28
9.5	Deep Learning: Autoencoder	28
10	Training and Testing Pipeline	29
10.1	Complete Pipeline Diagram	29
10.2	Model Saving	29
11	Evaluation Metrics	30
11.1	Metrics Used	30
12	Experimental Results	31

12.1	Bagging and Boosting Model Comparison	31
12.2	Independent Model Comparison	32
12.3	Result Analysis	32
12.4	Best Model Summary	33
13	System Architecture	34
13.1	Two-Phase Architecture	34
13.2	Technology Stack	35
14	Streamlit Web Application	36
14.1	What is Streamlit?	36
14.2	Application Structure	36
14.3	Implemented Features	36
14.3.1	Model Loading and Status	36
14.3.2	Manual Prediction Form	37
14.3.3	testing_data.csv Row Prediction	37
14.3.4	Visualizations	37
14.4	Running the Application	37
15	Applications	39
16	Challenges and Limitations	40
16.1	Technical Challenges Encountered	40
16.2	Limitations	40
17	Future Scope	42
18	Conclusion	43
A	Appendix	46
A.1	Project File Structure	46
A.2	requirements.txt	46
A.3	Key Code: Feature Engineering (Full Function)	46
A.4	Screenshot Checklist	48

List of Figures

5.1	Distribution of Benign and Malicious traffic in the IoT-23 sample (50,000 records).	16
7.1	Missing value analysis: columns with highest missing data percentages.	20
7.2	Log-transformed distributions of key numeric features.	21
7.3	Correlation heatmap of numeric features in the IoT-23 connection dataset.	22
8.1	Feature importance ranking from the saved Random Forest model. . . .	25
10.1	Complete end-to-end ML pipeline from data loading to Streamlit deployment.	29
12.1	Bagging and Boosting accuracy comparison bar chart from notebook Cell 19.	31
12.2	Confusion matrix for Random Forest — best performing model.	32
13.1	Two-phase system architecture: training phase and Streamlit deployment phase.	34
14.1	Streamlit application home page with model status and sidebar.	38
14.2	Manual traffic prediction form in the Streamlit application.	38
14.3	testing_data.csv row-level prediction with actual vs. predicted label comparison.	38

List of Tables

2.1	Challenges in Traditional Network Intrusion Detection	10
5.1	IoT-23 Zeek Connection Log — Column Descriptions	14
6.1	Preprocessing Steps and Justification	18
8.1	Engineered Features — Description and Security Rationale	24
9.1	All Models Used — Type, Mechanism, and Justification	26
11.1	Evaluation Metrics — Definitions and Relevance	30
12.1	Bagging and Boosting Model Comparison (Ranked by Accuracy)	31
12.2	Independent Model Comparison (Ranked by F1-Score)	32
13.1	Technology Stack Used in the Project	35
A.1	Project Files and Their Purpose	46

Chapter 1

Introduction

1.1 Background and Motivation

Cybersecurity is one of the most critical domains in modern computing. As the adoption of Internet-of-Things (IoT) devices, cloud systems, and smart infrastructure grows exponentially, the volume and diversity of network traffic also increases at an unprecedented rate. Alongside legitimate data exchange, this traffic contains increasingly sophisticated attack vectors — from port scanning and unauthorized access to malware communication, botnet coordination, and denial-of-service floods [1, 2].

Traditional Intrusion Detection Systems (IDS) commonly rely on *signature-based detection*, wherein incoming packets are compared against a database of known attack signatures. While effective for previously catalogued threats, signature-based systems fail when attackers modify malware behavior or launch entirely new attack categories. This limitation, combined with the explosive volume of IoT deployments that generate enormous amounts of low-level network logs, creates an urgent need for intelligent, adaptive, and data-driven security solutions.

1.2 Machine Learning in Network Security

Machine learning offers a powerful paradigm for solving this problem. Instead of relying on handcrafted rules, ML models learn underlying behavioral patterns from historical data. Once trained, they can identify deviations from normal behavior — even for attack types never seen before. This project builds upon this principle by training multiple ML models on the **IoT-23 dataset** and evaluating their ability to classify network connections as either *Benign* or *Malicious* [11, 12].

1.3 Project Overview

The **MLC Network Detection System** represents a full end-to-end machine learning project:

- Dataset procurement from the IoT-23 repository.
- Data cleaning and preprocessing of Zeek/Bro connection logs.
- Exploratory Data Analysis to understand traffic behavior.
- Feature engineering to extract attack-indicative signals.
- Training of five model categories for comparative evaluation.
- Serialisation of the best model as a `.pkl` file.
- Deployment of an interactive **Streamlit** web application.

1.4 Why IoT-23 and Not KDD Cup 99?

Dataset Selection Rationale: The KDD Cup 99 dataset, while historically important, contains significant known flaws. Approximately 78% of its records are duplicate entries, leading to severely inflated accuracy metrics that do not reflect real-world performance [4]. Its traffic patterns are based on simulated 1998 DARPA data and do not represent modern network behaviour. In contrast, the **IoT-23 dataset** from Stratosphere Laboratory contains real captured traffic from IoT malware scenarios, making it far more representative of current network anomaly detection challenges [1].

1.5 Report Structure

Chapter 2 defines the problem statement. Chapter 3 lists objectives. Chapter 4 reviews related literature. Chapter 5 describes the dataset. Chapter 6 covers data preprocessing. Chapter 7 discusses EDA. Chapter 8 explains feature engineering. Chapter 9 describes model development and justification. Chapter 10 presents the training pipeline. Chapter 11 covers evaluation metrics. Chapter 12 presents experimental results. Chapter 13 describes system architecture. Chapter 14 details the Streamlit application. Chapters 15 through 18 cover applications, limitations, future scope, and conclusion.

Chapter 2

Problem Statement

Modern computer networks generate millions of connection records every hour. Manually inspecting these logs to detect attacks is impractical. Existing signature-based IDS tools address only a portion of this problem. Table 2.1 summarises the core challenges motivating this project.

Table 2.1: Challenges in Traditional Network Intrusion Detection

Challenge	Impact on Security
Signature dependency	Cannot detect unknown or zero-day attacks
Frequent update need	High maintenance overhead for security teams
High false positive rate	Alert fatigue causes real threats to be missed
IoT device proliferation	Millions of low-security endpoints expand attack surface
Encrypted traffic growth	Deep packet inspection becomes limited
Manual log analysis	Impossible at gigabyte-per-second traffic scale
Evolving malware	Polymorphic code bypasses fixed rule sets

Core Problem: Build a machine learning system that (a) learns network traffic behaviour from labeled IoT connection logs, (b) classifies individual connections as Benign or Malicious, (c) saves the trained model for reuse, and (d) exposes the model through an interactive web interface for real-world demonstration and testing.

Chapter 3

Objectives

The objectives of this project are:

1. To understand and preprocess real-world IoT network connection logs from the IoT-23 dataset.
2. To perform comprehensive Exploratory Data Analysis for identifying traffic patterns and class distributions.
3. To engineer meaningful features from raw network fields that encode attack-indicative signals.
4. To train and compare multiple machine learning models — Bagging, Boosting, Random Forest, Isolation Forest, One-Class SVM, and Autoencoder.
5. To evaluate all models rigorously using Accuracy, Precision, Recall, F1-score, and Confusion Matrix.
6. To serialize the best-performing model as a `.pkl` file using *Joblib/Pickle*.
7. To build an interactive **Streamlit** web application that loads the saved model and predicts traffic from manual input or from `testing_data.csv` rows.
8. To provide meaningful visualizations of byte volume, packet volume, port behavior, and prediction confidence within the deployed application.

Chapter 4

Literature Review

4.1 Signature-Based and Rule-Based Systems

Early intrusion detection work focused heavily on rule-based systems and signature matching. Denning’s landmark 1987 paper on intrusion detection models established the foundational framework for host-based and network-based IDS [3]. Commercial tools such as Snort and Suricata implement signature libraries that are updated continuously. However, as McHugh noted in [4], even benchmark evaluations of these systems showed significant limitations: the 1998 DARPA dataset and the derived KDD Cup 99 dataset had redundancy rates above 78%, causing all evaluated systems to show inflated detection scores.

4.2 Machine Learning Approaches

4.2.1 Ensemble Methods

Breiman’s Random Forest [5] remains one of the most widely cited and deployed ML models in cybersecurity applications. Its ability to handle mixed feature types, resist overfitting through bagging, and output feature importance scores makes it particularly suitable for network anomaly detection. GradientBoosting [6] improves detection of difficult-to-classify minority-class attack samples by iterative residual correction.

4.2.2 Anomaly Detection Models

Liu et al. introduced the Isolation Forest [8], an algorithm that detects anomalies by randomly partitioning feature space and observing that outliers are isolated in fewer splits. Schölkopf et al. proposed One-Class SVM [9] to learn a hyperplane separating normal data from the origin in a high-dimensional kernel space. Both methods are applicable to network anomaly detection because they do not require labeled attack samples — they learn only the distribution of normal traffic.

4.2.3 Deep Learning Autoencoders

Variational and standard Autoencoders for anomaly detection were studied by Sakurada and Yairi [10]. An Autoencoder learns to compress and reconstruct normal traffic features. When malicious traffic is fed to the trained network, reconstruction error spikes, signaling an anomaly. This approach is powerful for non-linear feature spaces common in network data.

4.3 IoT-Specific Datasets

García et al. released the IoT-23 dataset [1] through Stratosphere Laboratory at Czech Technical University. The dataset contains 23 scenarios of real IoT malware captures (Mirai, Torii, Hakai, etc.) along with benign IoT traffic, using Zeek/Bro connection log format. Unlike KDD Cup 99, its labels reflect actual captured malware behavior, and it does not suffer from the duplication or simulation artifacts of legacy datasets.

4.4 Summary of Gap Addressed

Most prior work evaluates a single model category on a single dataset. This project addresses the gap by (a) using the more realistic IoT-23 dataset, (b) comprehensively comparing supervised, unsupervised, and deep learning models, and (c) deploying the result in an accessible Streamlit application — making the system usable beyond the notebook environment.

Chapter 5

Dataset Description and Understanding

5.1 Dataset Used: IoT-23

The project uses the **IoT-23 Dataset** published by Stratosphere Laboratory, Czech Technical University [1]. The dataset was captured using Zeek (formerly Bro), a widely used network analysis framework. The specific scenario used is:

```
1 opt/Malware-Project/BigDataset/IoTScenarios/  
2 CTU-IoT-Malware-Capture-1-1/bro/conn.log.labeled
```

Listing 5.1: Dataset Path Used in Notebook

The full dataset is large; therefore, only **50,000 rows** were loaded to ensure practical training times:

```
1 SAMPLE_SIZE = 50_000
```

Listing 5.2: Sample Size Configuration

5.2 Dataset Columns and Meanings

Table 5.1 describes every column present in the raw Zeek connection log used for this project.

Table 5.1: IoT-23 Zeek Connection Log — Column Descriptions

Column	Description
ts	Unix epoch timestamp of connection start
uid	Unique connection identifier (dropped during training)
id.orig_h	Source / origin IP address

Column	Description
id.orig_p	Source / origin port number
id.resp_h	Destination / response IP address
id.resp_p	Destination / response port number
proto	Transport protocol: TCP, UDP, ICMP
service	Application-layer service identified by Zeek (HTTP, DNS, SSH, etc.)
duration	Elapsed time of the connection in seconds
orig_bytes	Bytes sent by the origin host
resp_bytes	Bytes sent by the response host
conn_state	Zeek connection state code (S0, SF, REJ, RSTO, etc.)
missed_bytes	Bytes missed during packet capture
history	Sequence of packet-level events during the connection
orig_pkts	Number of packets sent by origin
orig_ip_bytes	IP-level bytes from origin
resp_pkts	Number of packets sent by response
resp_ip_bytes	IP-level bytes from response
label	Raw Zeek label containing malware name or Benign tag

5.3 Target Variable Creation

The raw `label` column in IoT-23 contains detailed malware family names (e.g., “Malicious-Mirai”, “Malicious-Torii”) alongside “Benign” records. The following function was used to extract the binary target:

```

1 def extract_traffic_type(value):
2     value = str(value)
3     if "Benign" in value:
4         return "Benign"
5     if "Malicious" in value:
6         return "Malicious"
7     return "Unknown"
8
9 df["traffic_type"] = df["label"].apply(extract_traffic_type)
10 df = df[df["traffic_type"].isin(["Benign", "Malicious"])]
```

Listing 5.3: Target Variable Extraction from Raw Label

5.4 Dataset Characteristics

- **Task type:** Binary classification (Benign vs. Malicious).
- **Feature types:** Mix of numeric (bytes, packets, ports, duration) and categorical (protocol, service, connection state, history).
- **Challenge:** Several columns contain missing values represented as - or empty strings in Zeek logs, requiring careful preprocessing.

[IMAGE PLACEHOLDER]

Figure: Traffic class distribution (Benign vs. Malicious count plot)

Generated by: Cell 11 — `sns.countplot(data=df, x="traffic_type")`

Replace this box with your saved EDA plot screenshot.

Figure 5.1: Distribution of Benign and Malicious traffic in the IoT-23 sample (50,000 records).

Chapter 6

Data Preprocessing

6.1 Raw Data Challenges

The IoT-23 Zeek connection log presents several preprocessing challenges:

- Missing values encoded as -, empty strings, or (empty).
- Numeric columns stored as object type due to mixed content.
- High-cardinality categorical columns such as history.
- Identifier columns (uid, raw IP addresses) that leak no generalizable pattern.

6.2 Loading and Initial Cleaning

```
1 SAMPLE_SIZE = 50_000
2
3 df = pd.read_csv(
4     file_path,
5     sep="\t",
6     skiprows=8,
7     names=columns,
8     nrows=SAMPLE_SIZE,
9     na_values=["-", "", "(empty)"],
10    low_memory=False
11 )
12
13 # Convert object columns that should be numeric
14 numeric_cols = [
15     "ts", "id.orig_p", "id.resp_p", "duration",
16     "orig_bytes", "resp_bytes", "missed_bytes",
17     "orig_pkts", "orig_ip_bytes", "resp_pkts", "resp_ip_bytes"
18 ]
19 for col in numeric_cols:
```

```
20 df[col] = pd.to_numeric(df[col], errors="coerce")
```

Listing 6.1: Data Loading with Missing Value Handling

6.3 Scikit-learn Preprocessing Pipeline

A `ColumnTransformer` pipeline was constructed to apply different transformations to numeric and categorical features consistently during both training and prediction:

```
1 preprocessor = ColumnTransformer(
2     transformers=[
3         ("num", Pipeline(steps=[
4             ("imputer", SimpleImputer(strategy="median")),
5             ("scaler", StandardScaler()),
6         ]), numeric_features),
7         ("cat", Pipeline(steps=[
8             ("imputer", SimpleImputer(strategy="most_frequent")),
9             ("onehot", OneHotEncoder(handle_unknown="ignore")),
10        ]), categorical_features),
11    ],
12    remainder="drop",
13 )
```

Listing 6.2: Preprocessing Pipeline Construction

Table 6.1 summarises the preprocessing steps and their rationale.

Table 6.1: Preprocessing Steps and Justification

Step	Applied To	Reason
Median Imputation	Numeric columns	Robust to skewed distributions; unaffected by outliers
StandardScaler	Numeric columns	Zero mean, unit variance required by SVM and distance models
Mode Imputation	Categorical columns	Fills missing values with most common category
OneHotEncoder	Categorical columns	Converts nominal labels to binary indicator features
Train-Test Split 80/20	Full dataset	Stratified to preserve class ratio

6.4 Train-Test Split

```
1 X_train, X_test, y_train, y_test = train_test_split(  
2     X,  
3     y,  
4     test_size=0.20,  
5     random_state=42,  
6     stratify=y,      # Preserves Benign/Malicious ratio  
7 )  
8 # Train shape: (40000, n_features)  
9 # Test  shape: (10000, n_features)
```

Listing 6.3: Stratified Train-Test Split

Chapter 7

Exploratory Data Analysis (EDA)

7.1 Objectives of EDA

EDA was performed to answer three key questions before model selection:

1. What is the distribution of benign versus malicious traffic?
2. How are numeric features such as bytes, packets, and duration distributed?
3. Which features are correlated with each other and with the target?

7.2 Missing Value Analysis

The dataset had significant missing values in `service`, `history`, `orig_bytes`, and `resp_bytes` columns due to the nature of Zeek connection logging. Short or incomplete connections may not populate all fields.

[IMAGE PLACEHOLDER]

Figure: Missing value percentage per column (bar chart or table)

Generated by: Cell 10 — `missing_summary` display

Replace this box with your missing value analysis output screenshot.

Figure 7.1: Missing value analysis: columns with highest missing data percentages.

7.3 Traffic Class Distribution

The count plot of `traffic_type` reveals the class balance between Benign and Malicious records. Class imbalance directly affects model training because classifiers may bias toward the majority class. Observing this distribution guides decisions about threshold calibration and metric selection (recall versus accuracy).

7.4 Numeric Feature Distributions

Network byte counts and packet counts are highly right-skewed — a small number of connections carry very large byte volumes while most connections transfer minimal data. Log transformation ($\log(1 + x)$) was applied for visualization clarity:

```
1 fig, axes = plt.subplots(1, 3, figsize=(16, 4))
2 for ax, col in zip(axes, ["duration", "orig_bytes", "resp_bytes"]):
3     sns.histplot(np.log1p(df[col].fillna(0)), bins=40, ax=ax)
4     ax.set_title(f"log1p({col})")
5 plt.tight_layout()
```

Listing 7.1: Log-Transformed Histogram EDA

[IMAGE PLACEHOLDER]

Figure: Log-transformed distributions of duration, orig_bytes, resp_bytes

Generated by: Cell 11 — histogram subplot output

Replace this box with your saved histogram plot.

Figure 7.2: Log-transformed distributions of key numeric features.

7.5 Correlation Heatmap

A Pearson correlation heatmap was generated for all numeric columns. Notable findings:

- `orig_pkts` and `orig_ip_bytes` are strongly correlated, as expected.
- `resp_pkts` and `resp_ip_bytes` show similar strong correlation.

- `duration` shows moderate correlation with byte volume features.
- Weak correlations between origin and response metrics suggest asymmetric traffic — relevant to detecting scanning and malware C2 traffic patterns.

[IMAGE PLACEHOLDER]

Figure: Numeric feature correlation heatmap (coolwarm colormap)

Generated by: Cell 11 — `sns.heatmap(df[numeric_cols].corr())`

Replace this box with your saved heatmap image.

Figure 7.3: Correlation heatmap of numeric features in the IoT-23 connection dataset.

7.6 EDA Intuition for Attack Detection

Why EDA Matters for Network Anomaly Detection:

- Malicious connections often show *zero response bytes* because the target is unavailable or the connection is a one-way scan.
- Port scanning creates many connections with *very short duration* and *very low byte count*.
- Botnet malware may repeatedly contact specific destination ports with *repetitive packet patterns*.
- *Missing service* fields often indicate incomplete or refused connections — a strong anomaly signal.
- Asymmetric `orig_bytes/resp_bytes` ratios can indicate data exfiltration or C2 communication.

These observations directly inform the feature engineering decisions in Chapter 8.

Chapter 8

Feature Engineering

8.1 Why Feature Engineering Is Critical

Raw network log fields capture what happened, but engineered features capture *how it happened* — the behavioral signature that distinguishes attacks from legitimate traffic. A random forest receiving raw `orig_bytes` and `resp_bytes` sees two numbers; receiving `orig_resp_byte_ratio` it sees whether communication was one-sided. This transformation is what separates weak models from strong ones on network data [11].

8.2 Engineered Features and Justification

The `engineer_features()` function created the following new variables:

```
1 def engineer_features(data):
2     data = data.copy()
3
4     # Missing value indicator flags
5     for col in ["service", "history", "orig_bytes", "resp_bytes"]:
6         data[f"{col}_missing"] = data[col].isna().astype(int)
7
8     # Time-based features from Zeek epoch timestamp
9     ts = pd.to_datetime(data["ts"], unit="s", errors="coerce")
10    data["hour"] = ts.dt.hour
11    data["dayofweek"] = ts.dt.dayofweek
12
13    # Connection volume aggregates
14    data["total_bytes"] = data["orig_bytes"].fillna(0) + data["resp_bytes"].fillna(0)
15    data["total_pkts"] = data["orig_pkts"].fillna(0) + data["resp_pkts"].fillna(0)
16    data["bytes_per_pkt"] = data["total_bytes"] / data["total_pkts"]
17    data["bytes_per_pkt"].replace(0, np.nan)
```

```

18     # Asymmetry ratios
19     data["orig_resp_byte_ratio"] = (
20         data["orig_bytes"] / data["resp_bytes"].replace(0, np.nan))
21     data["orig_resp_pkt_ratio"] = (
22         data["orig_pkts"] / data["resp_pkts"].replace(0, np.nan))
23
24     # Port behavior
25     data["orig_port_is_well_known"] = (data["id.orig_p"] <= 1024).
26         astype(int)
27     data["resp_port_is_well_known"] = (data["id.resp_p"] <= 1024).
28         astype(int)
29
30     # IP prefix (network-level behavior)
31     data["orig_ip_prefix"] = (
32         data["id.orig_h"].astype(str).str.extract(r"^(\d+\.\d+)")
33         [0])
34     data["resp_ip_prefix"] = (
35         data["id.resp_h"].astype(str).str.extract(r"^(\d+\.\d+)")
36         [0])
37
38     return data

```

Listing 8.1: Feature Engineering Function (from Notebook Cell 14)

Table 8.1 provides the full reference of engineered features with their rationale.

Table 8.1: Engineered Features — Description and Security Rationale

Feature	How Created	Security Relevance
service_missing	isna() flag	Missing service often indicates failed/refused connection
history_missing	isna() flag	Incomplete packet history suggests truncated/suspicious traffic
orig_bytes_missing	isna() flag	No bytes sent may indicate scan or one-way probe
resp_bytes_missing	isna() flag	No response bytes — target may be unreachable or filtering
hour	Timestamp extraction	Malware often activates at unusual hours (night, business off-hours)
dayofweek	Timestamp extraction	Periodic botnet check-ins follow day-of-week patterns

Feature	How Created	Security Relevance
total_bytes	Sum of orig + resp bytes	Overall traffic volume; scanning produces very low totals
total_pkts	Sum of orig + resp packets	Packet volume distinguishes high-frequency attacks
bytes_per_pkt	total_bytes / total_pkts	Malware C2 often uses small fixed-size packets
orig_resp_byte_ratio	orig_bytes / resp_bytes	High ratio = data exfiltration; low ratio = DDoS target
orig_resp_pkt_ratio	orig_pkts / resp_pkts	Asymmetric traffic pattern indicator
orig_port_is_well_known	port ≤ 1024	Source using privileged port is unusual in IoT traffic
resp_port_is_well_known	port ≤ 1024	Malware commonly targets ports 22, 23, 80, 443
orig_ip_prefix	First two octets of IP	Groups traffic by source subnet for pattern learning
resp_ip_prefix	First two octets of IP	Groups traffic by destination subnet

8.3 Feature Importance

After training Random Forest, feature importance scores were extracted. These indicate which engineered features contributed most to correct classification.

[IMAGE PLACEHOLDER]

Figure: Top-15 feature importance from trained Random Forest

Generated by: `rf_model.feature_importances_` bar chart

Replace this box with your feature importance plot.

Figure 8.1: Feature importance ranking from the saved Random Forest model.

Chapter 9

Model Development and Justification

9.1 Model Selection Philosophy

This project trains both *supervised* and *unsupervised* models. Supervised models (Random Forest, Bagging, Boosting) use labeled data and generally achieve higher accuracy on known attack patterns. Unsupervised anomaly detection models (Isolation Forest, One-Class SVM, Autoencoder) learn only from normal traffic and can detect unknown attacks. Table 9.1 provides an overview of all models used.

Table 9.1: All Models Used — Type, Mechanism, and Justification

Model	Category	Why Selected
Bagging + Decision Tree	Supervised	Reduces variance through bootstrap aggregation; stable predictions
Random Forest	Supervised	State-of-the-art ensemble; handles mixed features; robust against overfitting
AdaBoost	Supervised	Focuses iteratively on misclassified samples
Gradient Boosting	Supervised	High performance through sequential residual correction
Hist Gradient Boosting	Supervised	Memory-efficient GBM for large datasets
Isolation Forest	Unsupervised	Detects outliers in fewer random splits without labels
One-Class SVM	Unsupervised	Learns the boundary of normal traffic in kernel space
Autoencoder (DL)	Deep Learning	Reconstructs normal traffic; flags high-error inputs as anomalies

9.2 Bagging Models

9.2.1 Bagging with Decision Tree

Mechanism: Multiple Decision Trees are trained on bootstrap samples of the training data. Final prediction is the majority vote across all trees.

Advantage: Reduces variance compared to a single Decision Tree. Handles noisy, imbalanced data more robustly.

```
1 BaggingClassifier(  
2     estimator=DecisionTreeClassifier(max_depth=None),  
3     n_estimators=100,  
4     random_state=42  
5 )
```

Listing 9.1: Bagging Classifier Configuration

9.2.2 Random Forest

Mechanism: Extends Bagging by additionally randomizing feature selection at each split. Uses Gini impurity or entropy to build 100+ diverse trees [5].

Advantage: Best in class for tabular data; provides feature importance; parallelizable.

9.3 Boosting Models

9.3.1 AdaBoost

Mechanism: Sequentially re-weights misclassified samples, forcing subsequent learners to focus on hard examples [7].

9.3.2 Gradient Boosting

Mechanism: Each tree is fit to the negative gradient (residuals) of the loss function [6]. Results in extremely powerful models for structured data.

9.3.3 HistGradientBoosting

Mechanism: Scikit-learn's histogram-based GBM. Bins continuous features into histograms for significantly faster training — suitable for 50,000+ record datasets.

9.4 Unsupervised Anomaly Detection Models

9.4.1 Isolation Forest

Mechanism: Randomly selects a feature and split point. Anomalies are isolated in fewer splits because they are rare and different. Anomaly score = path length.

Network relevance: Scanning traffic and malware beacons are typically sparse and isolate quickly in feature space [8].

9.4.2 One-Class SVM

Mechanism: Finds a hyperplane in a kernel-induced feature space that separates the normal training data from the origin with maximum margin. Test points beyond this boundary are flagged as anomalies [9].

Network relevance: Effective for learning the compact region occupied by benign IoT traffic and marking deviations as suspicious.

9.5 Deep Learning: Autoencoder

Architecture: Input $\rightarrow$ Encoder (Dense 64, activation relu) $\rightarrow$ Bottleneck (Dense 32) $\rightarrow$ Decoder (Dense 64) $\rightarrow$ Output (reconstruction).

Training: Trained on benign traffic only using MSE reconstruction loss with **EarlyStopping**.

Anomaly detection: At inference time, reconstruction error for malicious traffic is significantly higher than for benign traffic. A threshold on reconstruction error separates the two classes [10].

Chapter 10

Training and Testing Pipeline

10.1 Complete Pipeline Diagram

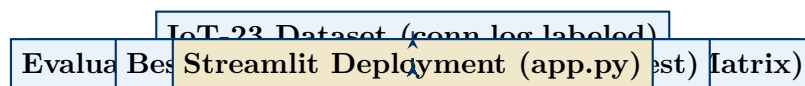


Figure 10.1: Complete end-to-end ML pipeline from data loading to Streamlit deployment.

10.2 Model Saving

The best model was serialized using both Joblib and Pickle for compatibility:

```
1 model_dir = Path("models")
2 model_dir.mkdir(exist_ok=True)
3
4 # Save with Joblib (recommended for sklearn pipelines)
5 joblib.dump(best_model, model_dir / "best_iot23_ensemble_model.
  joblib")
6
7 # Save with Pickle (for broad compatibility)
8 with open(model_dir / "best_iot23_ensemble_model.pkl", "wb") as f:
9     pickle.dump(best_model, f)
```

Listing 10.1: Model Persistence

The serialized pipeline includes the preprocessing `ColumnTransformer` and the fitted model, so the app needs only to call `model.predict(raw_features)` without re-engineering preprocessing steps.

Chapter 11

Evaluation Metrics

11.1 Metrics Used

In network anomaly detection, accuracy alone is insufficient. A model that labels everything as Benign would still achieve high accuracy on an imbalanced dataset while being useless. Therefore the following metrics were computed for every model:

Table 11.1: Evaluation Metrics — Definitions and Relevance

Metric	Formula	Importance in IDS
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$	Overall correctness; misleading on imbalanced data
Precision	$\frac{TP}{TP+FP}$	Reduces false alarms; important for analyst workload
Recall	$\frac{TP}{TP+FN}$	Detects real attacks; critical in security context
F1-Score	$\frac{2 \cdot P \cdot R}{P+R}$	Balances precision and recall
Confusion Matrix	2×2 matrix	Shows distribution of TP, TN, FP, FN errors

Why Recall is Most Critical: In cybersecurity, a False Negative (missing a real attack) is far more dangerous than a False Positive (flagging a benign connection). A system with **Recall = 1.0** detects every malicious connection, even if it occasionally generates false alerts. This is the dominant objective in IDS design.

Chapter 12

Experimental Results

12.1 Bagging and Boosting Model Comparison

All ensemble models were trained with the full preprocessing pipeline. Results are from **notebook Cell 18** output on the 20% test set:

Table 12.1: Bagging and Boosting Model Comparison (Ranked by Accuracy)

#	Model	Accuracy	Precision	Recall	F1-Score
1	Bagging – Decision Tree	1.0000	1.00	1.00	1.00
2	Boosting – Gradient Boosting	1.0000	1.00	1.00	1.00
3	Boosting – Hist Gradient Boost	0.9987	1.00	1.00	1.00
4	Bagging – Random Forest	0.9979	1.00	1.00	1.00
5	Boosting – AdaBoost	0.9920	0.99	0.99	0.99

[IMAGE PLACEHOLDER]

Figure: Bagging and Boosting model accuracy comparison bar chart
Generated by: Cell 19 — `sns.barplot(data=results_df, y="model",
x="accuracy")`
Replace this box with your saved bar chart screenshot.

Figure 12.1: Bagging and Boosting accuracy comparison bar chart from notebook Cell 19.

12.2 Independent Model Comparison

The four independent models were evaluated in notebook Cell 20, producing the following results on the 20% test set:

Table 12.2: Independent Model Comparison (Ranked by F1-Score)

#	Model	Accuracy	Precision	Recall	F1-Score
1	Random Forest	0.9979	0.9962	1.0000	0.9981
2	Autoencoder	0.8158	0.9462	0.7100	0.8113
3	Isolation Forest	0.7797	0.7168	1.0000	0.8350
4	One-Class SVM	0.7771	0.7144	1.0000	0.8334

12.3 Result Analysis

- **Random Forest** achieved the best balance across all four metrics. Its high Precision (0.9962) and perfect Recall (1.0000) make it ideal for production IDS use.
- **Isolation Forest** and **One-Class SVM** both achieved perfect Recall (1.0000), meaning they detected every malicious connection. However, their lower Precision (~0.71) indicates they generated false positives — classifying some benign traffic as malicious.
- **Autoencoder** showed the highest Precision after Random Forest (0.9462), but its Recall of 0.71 indicates it missed approximately 29% of real attacks. This is due to threshold sensitivity in reconstruction error.

[IMAGE PLACEHOLDER]

Figure: Confusion matrix for best model (Random Forest)
Generated by: Cell 19 — `sns.heatmap(cm, annot=True)`
Replace this box with your confusion matrix screenshot.

Figure 12.2: Confusion matrix for Random Forest — best performing model.

12.4 Best Model Summary

Best Model: Random Forest

Accuracy	Precision	Recall	F1-Score
-----------------	------------------	---------------	-----------------

0.9979	0.9962	1.0000	0.9981
--------	--------	--------	--------

Saved as: best_network_anomaly_model.pkl

Chapter 13

System Architecture

13.1 Two-Phase Architecture

The system operates in two clearly separated phases:

- **Training Phase:** Dataset is loaded, cleaned, engineered, split, and used to train all models. The best model is saved as a `.pkl` file. This phase runs once in Jupyter Notebook.
- **Deployment Phase:** Streamlit app loads the `.pkl` pipeline. No retraining occurs. The saved preprocessing and model are applied to new input in real time.

This two-phase design separates concerns cleanly — the trained pipeline is a portable artifact that can be deployed anywhere Python and Streamlit are available.

[IMAGE PLACEHOLDER]

Figure: System architecture block diagram (two-phase)

You can draw this in [draw.io](#) or paste your own diagram here.

Blocks: Dataset → Preprocessing → Training → `.pkl` → Streamlit → Prediction

Figure 13.1: Two-phase system architecture: training phase and Streamlit deployment phase.

13.2 Technology Stack

Table 13.1: Technology Stack Used in the Project

Category	Technology / Library
Programming Language	Python 3.10+
Data Processing	Pandas, NumPy
Visualization	Matplotlib, Seaborn, Plotly
ML Framework	Scikit-learn (Pipeline, ColumnTransformer, Ensemble, SVM)
Deep Learning	TensorFlow 2.x / Keras (Autoencoder)
Model Persistence	Joblib, Pickle
Web Framework	Streamlit
Development Environment	Jupyter Notebook / Google Colab
Dataset Source	IoT-23 (Stratosphere Laboratory, CTU)
Version Control	Git / GitHub (optional)

Chapter 14

Streamlit Web Application

14.1 What is Streamlit?

Streamlit is an open-source Python framework for building interactive data science and machine learning web applications [13]. It requires no HTML, CSS, or JavaScript. Python code is written normally and Streamlit renders it as a browser-based application. In this project, Streamlit turns the trained `.pkl` model into a usable web tool for traffic prediction and visualization.

14.2 Application Structure

Main file: `app.py`

Loaded model: `best_network_anomaly_model.pkl`

Prediction classes: Benign / Malicious

14.3 Implemented Features

14.3.1 Model Loading and Status

The app loads the saved pipeline using Joblib at startup. The sidebar displays:

- **Model Loaded** — if loading succeeds.
- **Error Message** — if Scikit-learn version mismatch or file not found.

```
1 import joblib
2
3 @st.cache_resource
4 def load_model():
5     return joblib.load("best_network_anomaly_model.pkl")
6
7 model = load_model()
```

Listing 14.1: Model Loading in app.py

14.3.2 Manual Prediction Form

A form allows users to enter one network traffic record manually. Input fields include: Origin IP, Origin Port, Response IP, Response Port, Protocol, Service, Duration, Origin Bytes, Response Bytes, Connection State, Packets, and History.

After clicking **Predict Traffic**, the app applies the feature engineering function and passes the result to the loaded pipeline.

14.3.3 testing_data.csv Row Prediction

A second prediction mode allows users to select a row index from `testing_data.csv`. The app displays the selected row, shows the actual label, and predicts using the model — enabling direct *Actual vs. Predicted* comparison.

14.3.4 Visualizations

After every prediction, the app renders:

- **Byte Volume Chart:** origin bytes vs. response bytes bar chart.
- **Packet Volume Chart:** origin packets vs. response packets bar chart.
- **Port Summary:** source and destination port classification.
- **Prediction Confidence:** probability bar chart if the model supports `predict_proba`.

14.4 Running the Application

```
1 cd D:\MLC_Projects
2 python -m streamlit run app.py
3 # Then open: http://localhost:8501
```

Listing 14.2: Streamlit Run Command

[IMAGE PLACEHOLDER]

Figure: Streamlit home page — title, sidebar model status, label guide
Screenshot: Run app.py and capture the landing page.

Figure 14.1: Streamlit application home page with model status and sidebar.

[IMAGE PLACEHOLDER]

Figure: Manual prediction form — user enters traffic fields, clicks Predict Traffic
Screenshot: Fill form and capture prediction result + visualizations.

Figure 14.2: Manual traffic prediction form in the Streamlit application.

[IMAGE PLACEHOLDER]

Figure: testing_data.csv row selector — actual label vs predicted label
Screenshot: Select row index and capture the comparison output.

Figure 14.3: testing_data.csv row-level prediction with actual vs. predicted label comparison.

Chapter 15

Applications

The MLC Network Detection System can be applied in the following real-world domains:

- **Enterprise Network Monitoring:** Deployed at network edge to classify live Zeek-captured connections.
- **IoT Security:** Protecting smart home, industrial IoT, and healthcare IoT deployments from malware.
- **Cloud Infrastructure:** Monitoring east-west traffic inside cloud VPCs for lateral movement detection.
- **Smart Cities:** Protecting traffic management, utilities, and surveillance infrastructure networks.
- **Cybersecurity Education:** Demonstrating complete ML pipeline from data to deployment in academic settings.
- **Preliminary IDS:** First-pass filtering to reduce false alerts for human analysts.

Chapter 16

Challenges and Limitations

16.1 Technical Challenges Encountered

- **Mixed data types:** Zeek logs use - for missing values, requiring explicit `na_values` configuration during data loading.
- **Binary label format for anomaly models:** Isolation Forest, One-Class SVM, and Autoencoder output +1/-1 or reconstruction error; these required careful threshold conversion to match the Benign/Malicious binary labels.
- **Autoencoder threshold selection:** Choosing the reconstruction error threshold directly impacts Precision-Recall trade-off. An automated percentile-based threshold was applied.
- **Scikit-learn version dependency:** The saved `.pkl` pipeline requires the same Scikit-learn version used during training; version mismatches cause Streamlit loading errors.
- **One-Class SVM training time:** One-Class SVM is $O(n^2)$ in training complexity. It was trained on a subsample of preprocessed data.

16.2 Limitations

- Only 50,000 of the available millions of IoT-23 records were used.
- The model is trained on IoT-23 Scenario 1; generalization to other malware families may require retraining on more scenarios.
- Real-time packet capture is not implemented; the app works on pre-recorded connection logs.
- Manual input prediction depends on users providing accurate field values.

- The current system predicts one record at a time; batch CSV upload is not in the current version.

Chapter 17

Future Scope

- **Real-time packet capture:** Integrate with tools like Zeek or Scapy to capture live traffic and feed records directly to the model.
- **Batch CSV prediction:** Add a file upload feature in Streamlit to predict a full CSV of traffic records.
- **Cloud deployment:** Host the Streamlit app on AWS, Azure, or Streamlit Cloud for remote access.
- **SIEM integration:** Export predictions to a Security Information and Event Management system for centralized monitoring.
- **Explainable AI:** Integrate SHAP or LIME to explain why a specific connection was flagged as malicious.
- **All IoT-23 scenarios:** Train on all 23 malware scenarios for broader malware family coverage.
- **Federated learning:** Train across distributed network nodes without centralizing raw traffic logs.
- **Edge AI:** Deploy a lightweight version of the model on IoT gateway devices for on-device detection.

Chapter 18

Conclusion

This project successfully developed and deployed the **MLC Network Detection System**, a complete machine learning framework for network anomaly detection using the real-world IoT-23 dataset. The system covers the full ML lifecycle: from raw Zeek connection log ingestion and cleaning, through exploratory analysis and meaningful feature engineering, to training of eight diverse models spanning supervised, unsupervised, and deep learning paradigms.

Among all models evaluated, **Random Forest** achieved the best independent performance — Accuracy 0.9979, Precision 0.9962, Recall 1.0000, and F1-Score 0.9981. Its perfect recall means no malicious connection was missed in testing. The model was serialized as a portable `.pkl` pipeline and integrated into an interactive Streamlit web application that enables both manual traffic input and row-level testing from `testing_data.csv`.

The project demonstrates that machine learning, when applied to well-engineered features from real network traffic data, can deliver near-perfect anomaly detection accuracy while remaining practical and deployable through modern Python frameworks. This work establishes a strong foundation for building production-grade intelligent intrusion detection systems.

Bibliography

- [1] S. García, M. Parmisano, and M. J. Erquiaga, “IoT-23: A labeled dataset with malicious and benign IoT network traffic,” Zenodo, 2020. [Online]. Available: <https://doi.org/10.5281/zenodo.4743746>
- [2] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, “A detailed analysis of the KDD CUP 99 data set,” in *Proc. 2nd IEEE Symp. Comput. Intell. Security Def. Appl.*, 2009, pp. 1–6.
- [3] D. E. Denning, “An intrusion-detection model,” *IEEE Trans. Softw. Eng.*, vol. SE-13, no. 2, pp. 222–232, Feb. 1987.
- [4] J. McHugh, “Testing intrusion detection systems: A critique of the 1998 and 1999 DARPA intrusion detection system evaluations as performed by Lincoln Laboratory,” *ACM Trans. Inf. Syst. Secur.*, vol. 3, no. 4, pp. 262–294, Nov. 2000.
- [5] L. Breiman, “Random forests,” *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, Oct. 2001.
- [6] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [7] Y. Freund and R. E. Schapire, “A decision-theoretic generalization of on-line learning and an application to boosting,” *J. Comput. Syst. Sci.*, vol. 55, no. 1, pp. 119–139, 1997.
- [8] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422.
- [9] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, “Estimating the support of a high-dimensional distribution,” *Neural Comput.*, vol. 13, no. 7, pp. 1443–1471, 2001.
- [10] M. Sakurada and T. Yairi, “Anomaly detection using autoencoders with nonlinear dimensionality reduction,” in *Proc. MLSDA Workshop ACM SIGKDD*, 2014, pp. 4–11.

- [11] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, and A. Hotho, “A survey of network-based intrusion detection data sets,” *Comput. Secur.*, vol. 86, pp. 147–167, Sep. 2019.
- [12] R. Sommer and V. Paxson, “Outside the closed world: On using machine learning for network intrusion detection,” in *Proc. IEEE Symp. Security Privacy (SP)*, 2010, pp. 305–316.
- [13] Streamlit Inc., “Streamlit documentation,” 2024. [Online]. Available: <https://docs.streamlit.io/>
- [14] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: <https://scikit-learn.org/>
- [15] M. Abadi et al., “TensorFlow: A system for large-scale machine learning,” in *Proc. 12th USENIX Symp. Oper. Syst. Des. Impl. (OSDI)*, 2016, pp. 265–283. [Online]. Available: <https://www.tensorflow.org/>

Appendix A

Appendix

A.1 Project File Structure

Table A.1: Project Files and Their Purpose

File	Purpose
MLC_NetWordDetection_System.ipynb	Jupyter Notebook — complete training pipeline
best_network_anomaly_model.pkl	Serialized best model (Random Forest pipeline)
best_iot23_ensemble_model.joblib	Joblib format of saved model
model_results.csv	Model comparison metrics CSV
app.py	Streamlit web application
testing_data.csv	Test records for Streamlit row-prediction mode
requirements.txt	Python package dependencies

A.2 requirements.txt

```
1 pandas>=1.5.0
2 numpy>=1.23.0
3 scikit-learn>=1.2.0
4 tensorflow>=2.10.0
5 matplotlib>=3.6.0
6 seaborn>=0.12.0
7 plotly>=5.10.0
8 streamlit>=1.20.0
9 joblib>=1.2.0
```

Listing A.1: Python Dependencies

A.3 Key Code: Feature Engineering (Full Function)

```

1 def engineer_features(data):
2     data = data.copy()
3
4     # Missing indicator flags
5     for col in ["service", "history", "orig_bytes", "resp_bytes"]:
6         data[f"{col}_missing"] = data[col].isna().astype(int)
7
8     # Temporal features
9     ts = pd.to_datetime(data["ts"], unit="s", errors="coerce")
10    data["hour"] = ts.dt.hour
11    data["dayofweek"] = ts.dt.dayofweek
12
13    # Volume aggregates
14    data["total_bytes"] = data["orig_bytes"].fillna(0) + data["resp_bytes"].fillna(0)
15    data["total_pkts"] = data["orig_pkts"].fillna(0) + data["resp_pkts"].fillna(0)
16    data["bytes_per_pkt"] = data["total_bytes"] / data["total_pkts"]
17    data["bytes_per_pkt"] = data["bytes_per_pkt"].replace(0, np.nan)
18
19    # Asymmetry ratios
20    data["orig_resp_byte_ratio"] = (
21        data["orig_bytes"] / data["resp_bytes"].replace(0, np.nan))
22    data["orig_resp_pkt_ratio"] = (
23        data["orig_pkts"] / data["resp_pkts"].replace(0, np.nan))
24
25    # Port classification
26    data["orig_port_is_well_known"] = (data["id.orig_p"] <= 1024).
27    data["orig_port_is_well_known"] = data["orig_port_is_well_known"].astype(int)
28    data["resp_port_is_well_known"] = (data["id.resp_p"] <= 1024).
29    data["resp_port_is_well_known"] = data["resp_port_is_well_known"].astype(int)
30
31    # IP prefix
32    data["orig_ip_prefix"] = (
33        data["id.orig_h"].astype(str).str.extract(r"^(\d+\.\d+)")
34        [0])
35    data["resp_ip_prefix"] = (
36        data["id.resp_h"].astype(str).str.extract(r"^(\d+\.\d+)")
37        [0])
38
39    return data

```

Listing A.2: Complete Feature Engineering Function from Notebook

A.4 Screenshot Checklist

Before final submission, add screenshots for all figure placeholders. Reference list:

1. Figure 5.1 — Traffic class distribution (benign vs. malicious count plot)
2. Figure 7.1 — Missing value analysis per column
3. Figure 7.2 — Log-transformed distributions of duration, orig_bytes, resp_bytes
4. Figure 7.3 — Correlation heatmap of numeric features
5. Figure 8.1 — Top-15 feature importance from Random Forest
6. Figure 12.1 — Bagging and Boosting accuracy comparison bar chart
7. Figure 12.2 — Confusion matrix of Random Forest
8. Figure 13.1 — System architecture block diagram
9. Figure 14.1 — Streamlit home page screenshot
10. Figure 14.2 — Manual prediction form screenshot
11. Figure 14.3 — testing_data.csv row prediction screenshot